



Next State 계산

종류

프로젝트 정책

파라미터 접미사 네이밍 규칙

접미사	의미	시간과의 관계	일반적인 단위
Rate	단위 시간당 변화 속도	보통 $\times \Delta t$ 하여 실제 변화량 계산	/s, State/s, 1/s 등
Factor	기본 입력·계산·변환 결과를 조절하는 계수	시간과 직접적인 관계 없음	보통 무차원. 단, 물리량 변환 시 단위를 가 질 수도 있음
Weight	특정 texel·방향·인접 관계 에서 적용되는 국소 가중치	시간과 직접적인 관계 없음	보통 무차원 [0, 1]

전체 수식

$$State_i^{t+1} = clamp(State_i^t + Input_i + Transport_i - Decay_i, 0, Capacity_i)$$

- $Input$ (외부요인)은 음수일 수도 있다.
- $Transport = Incoming - Outgoing$
- Capacity로 Clamp하는 것까지 포함하면 아래의 식이 된다.

$$S_i^{t+1} = S_i^t + Input_i + Transport_i - Decay_i$$

- State는 다음 범위를 가진다.

$$0 \leq State \leq StateCapacity$$

시간 처리 기준

Input은 프레임 단위의 Discrete Event로 즉시 반영하고,
Transport와 Decay는 시간 기반 변화이므로 Δt 를 곱하여 계산한다.

항	처리 방식	Δt 반영	의미
Input	Discrete Event	X	접촉 이벤트 발생 시 해당 프레임에 즉시 반영
Transport	Continuous Update	O	시간 경과에 따른 상태 이동량으로 계산
Decay	Continuous Update	O	시간 경과에 따른 상태 감소량으로 계산

1. Input

*접촉 이벤트 발생 시 한 번에 반영되는 **Discrete 입력**이므로 Δt 는 곱하지 않는다.

$$Input_i = Strength \times ContactWeight_i \times InputFactor_i$$

$$ContactWeight_i = Falloff \left(\frac{Distance_i}{Radius} \right)$$

항	의미	범위 / 출처
Strength	외부 Source 자체의 입력 세기. 물, 열, 진흙 등의 입력 강도를 나타냄.	Contact Input Data
ContactWeight	접촉 영역 내에서 해당 texel이 입력을 얼마나 받는지 나타내는 가중치.	[0,1]
InputFactor	해당 재질이 외부 입력을 State로 얼마나 받아들이는지 나타내는 계수.	SRProfile, [0,n]

Raycast

→ World Hit Position + Hit Triangle

Hit Triangle의 Simulation UV 계산

→ 영향 받을 후보 texel 영역 찾기

각 후보 texel의 실제 Surface Position 구하기

→ Hit Position과의 거리 계산

Distance / Radius

→ ContactWeight

2. Transport (Incoming / Outgoing)

Transport

$$Transport_i = Incoming_i - Outgoing_i$$

$$Incoming_i = \sum_j Flux_{j \rightarrow i}$$

$$Outgoing_i = \sum_j Flux_{i \rightarrow j}$$

항	의미	범위 / 출처
Incoming	인접 texel들로부터 현재 texel로 들어오는 Flux의 총합.	- 인접 Texel Flux 합산 [0, n]
Outgoing	현재 texel에서 인접 texel들로 나가는 Flux의 총합. 보유한 State를 초과하지 않도록 제한됨.	- 현재 Texel Flux 합산 [0, Current_State]
Transport	Incoming - Outgoing 으로 계산되는 현재 texel의 순 상태 이동량.	[-Current_State, n]

항	의미	범위 / 출처
	양수이면 State가 증가하고, 음수이면 감소함.	

Flux

$$RawFlux_{i \rightarrow j} = (SaturationDrive_{i \rightarrow j} \cdot SaturationTransferRate + GeometryDrive_{i \rightarrow j} \cdot GeometryTransferRate) \cdot TransferWeight_{i \rightarrow j} \cdot \Delta t$$

포화도 차이에 의한 전달과 Geometry에 의한 전달을 각각 독립적으로 계산한다.

$$SaturationDrive_{i \rightarrow j} = \max(Saturation_i - Saturation_j, 0)$$

$$GeometryDrive_{i \rightarrow j} = HeightDrive_{i \rightarrow j} \cdot DirectionDrive_{i \rightarrow j}$$

Decay를 우선적으로 처리하고, 남은 State가 전체 RawFlux보다 부족할 경우 모든 Outgoing Flux를 동일한 비율로 감소시킨다.

이 배율을 alpha라고 한다.

$$Flux_{i \rightarrow j} = \alpha_i \cdot RawFlux_{i \rightarrow j}$$

항	의미	범위 / 출처
SaturationDrive	보내는 texel과 받는 texel 사이의 Saturation 차이에 의한 전달 구동력. 보내는 쪽의 Saturation이 더 높을 때만 발생함.	계산값, [0,1]
SaturationTransferRate	Saturation 차이에 의해 State가 전달되는 기본 속도.	SRProfile, [0,n]
GeometryDrive	높이 차이와 중력·표면 방향 등 Geometry에 의해 발생하는 전달 구동력.	Geometry 기반
GeometryTransferRate	Geometry에 의해 State가 전달되는 기본 속도.	SRProfile, [0,n]
TransferWeight	거리, 표면 방향 유사도, Material Boundary 등 해당 인접 관계에서 전달이 얼마나 잘 이루어지는지를 나타내는 가중치.	Geometry / Material 기반
alpha	Decay 처리 후 남은 State가 전체 RawFlux보다 부족한 경우 모든 Outgoing Flux를 동일 비율로 줄이기 위한 배율.	계산값, [0,1]

이때 Saturation(포화도)은 State와 StateCapacity에 의해 유도되는 파생값이다. 초안은 포화도가 아닌 인접 텍셀간 State 값의 차이를 그대로 사용했으나 Capacity 파라미터가 도입되면서 정규화가 필요해졌다.

$$Saturation = \frac{State}{StateCapacity}$$

TransferWeight: 전달 가중치

- Geometry와 SRProfile 경계에 의해 전달 가중치가 변화한다.
- GeometryDrive 가 이동을 발생시키는 방향·구동력을 결정한다면, TransferWeight 는 해당 인접 texel 관계에서 실제 전달이 얼마나 잘 이루어지는지를 보정한다.

$$TransferWeight_{i \rightarrow j} = W_{distance} \cdot W_{normal} \cdot W_{curvature} \cdot W_{profileBoundary}$$

Weight	의미	계산 기준
DistanceWeight	실제 표면상 가까운 이웃으로 전달이 잘 되도록 보정.	Surface Distance
NormalWeight	표면 방향이 비슷한 영역 사이에서 전달이 잘 되도록 보정.	Surface Normal
CurvatureWeight	국소적인 홈이나 요철에 의해 State가 붙잡히거나 이동이 억제되는 정도를 반영.	Curvature / Concavity
ProfileBoundaryWeight	서로 다른 SRProfile 영역 사이에서 State가 얼마나 잘 전달되는지 조절. 동일한 SRProfile 사이에서는 기본적으로 1.0.	SRProfile Boundary

▼ 구버전

시간에 따른 연속 변화이므로 Δt 를 곱한다.

$$RawFlux_{i \rightarrow j} = StateDifference_{i \rightarrow j} \times TransferRate \times TransferWeight_{i \rightarrow j} \times \Delta t$$

Decay를 우선적으로 처리하고, 가진 state 상태값이 부족하면 배율을 줄여야한다. 그 배율을 α_i 라고 한다.

$$Flux_{i \rightarrow j} = \alpha_i \times RawFlux_{i \rightarrow j}$$

$$\alpha_i = \min \left(1, \frac{\max(State_i - Decay_i, 0)}{\sum_j RawFlux_{i \rightarrow j}} \right)$$

항	의미	범위 / 출처
StateDifference	보내는 texel과 받는 texel 사이의 State 차이. 보내는 쪽의 값이 더 높을 때만 전달이 발생하도록 음수 값을 제거함.	계산값, [0, n]
TransferRate	현재 texel이 보유한 State를 인접 texel로 전달하는 기본 속도.	SRProfile, [0, n]
TransferWeight	특정 인접 texel 방향으로 상태가 얼마나 잘 전달되는지 나타내는 가중치.	Geometry 기반
alpha	Decay를 우선 반영한 뒤 남은 State가 전체 RawFlux 보다 부족할 경우, 모든 Outgoing Flux를 동일한 비율로 줄이기 위한 배율.	계산값, [0, 1]

TransferWeight: 전달 가중치

- Geometry, Seam에 의해 전달 가중치가 변화

$$TransferWeight_{i \rightarrow j} = W_{distance} W_{normal} W_{height} W_{direction} W_{boundary}$$

Weight	의미	계산 기준
DistanceWeight	실제 표면상 가까운 이웃으로 전달이 잘 되도록 보정.	Surface Distance
NormalWeight	표면 방향이 비슷한 영역 사이에서 전달이 잘 되도록 보정.	Surface Normal

Weight	의미	계산 기준
HeightWeight	높이 차이와 국소 요철에 따라 전달을 보정.	Height / Virtual Height
DirectionWeight	중력 등의 선호 방향을 반영하여 전달을 보정.	Gravity, Surface Direction
BoundaryWeight	서로 다른 Material 영역 사이의 전달 정도를 조절.	Material Boundary



헛갈릴 수 있는 3가지 정리

- **SaturationDrive** : 상태 차이에 의해 이동하려는 힘
- **GeometryDrive** : 높이·중력 때문에 이동하려는 힘
- **TransferWeight** : 그 이동이 해당 이웃 관계를 실제로 얼마나 잘 통과할 수 있는가

구버전에서는 **TransferWeight** 내부에 Geometry가 포함이 되었었다.
그런데 Geometry에 의한 영향 정도를 별도로 파라미터로 조정할 수 있으면 좋을 것 같아서 현재 버전으로 수정되었다.

3. Decay

시간에 따른 연속 변화이므로 Δt 를 곱한다.

$$Decay_i = DecayRate \times (1 - ConcavityWeight_i \times CavityRetentionFactor) \times \Delta t$$

계산된 **Decay** 가 현재 **State** 보다 큰 경우에는 **State** 값으로 제한하여, 한 번의 갱신에서 보유한 상태량보다 많이 감소하지 않도록 한다.

$$Decay_i = \min(Decay_i, State_i)$$

항	의미	범위 / 출처
DecayRate	해당 상태가 자연스럽게 감소하는 기본 속도.	SRProfile, [0,n]
ConcavityWeight	현재 texel이 국소적으로 얼마나 오목한지를 나타내는 값. 값이 클수록 오목한 영역임.	Geometry Data, [0,1]
ConcavityRetentionFactor	해당 상태가 오목한 영역에서 얼마나 오래 남는지를 결정하는 계수.	SRProfile, [0,1]

Convavity

$$k_1, k_2 \xrightarrow{\text{preprocess}} \textit{ConcavityWeight}$$

전처리 계산:

```

k1, k2
↓
Mean / Gaussian 또는 직접 판정
↓
ConcavityWeight

```

curvature는 전처리 중간값, 런타임에는 최종적으로 저장된 **ConcavityWeight** 만 읽는다.

Compute Shader 계산 과정

```

thread i
↓
Current 주변을 읽음
↓
Incoming 계산
Outgoing 계산
Decay 계산
Input 계산
↓
Next[i] 하나만 씀

```

- 각 항의 값을 계산할 때, 오직 현재의 상태값들(버퍼1)만 보고 계산하므로 각 항에 대해 병렬 처리가 가능 (각 텍셀별로 incoming에 대해서 재계산하는걸 택하면 그럼)
 - 근데 그렇다고 해서 한 텍셀의 각 항까지 스레드 하나씩이 맡는건 아닌 것 같음.
 - locality도 생각해서, 스레드 하나가 텍셀 하나를 담당하도록 한다
- Gather 방식
 - 스레드는 각 texel을 담당
 - 오직 자신의 `Next[i]` 만 write한다.
 - → race condition 없음.
- ping-pong buffer
 - 버퍼1: read용
 - 버퍼2: write용
 - 다음 상태 값을 계산하는 동안 read용이 필요.

음수 상태값 방지

$$Decay_i + Outgoing_i \leq S_i^t$$

- Decay는 기본적으로 보장
- 주변 8텍셀 각각에 대한 outgoing을 연산해야한다.
- Outgoing은 현재 state값이 부족하면, 비율로 줄인다.
 - 이 비율값 `alpha` 는 별도로 구해야한다.

이거 때문에 `Incoming` 항을 계산할 때, 주변 8텍셀 각각의 alpha를 알아야하는 문제가 생긴다!!!

만약 write & read 하지 않고 그냥 뺄셈으로 재계산을 한다면, 무려 $8 \times 8 = 64$ 텍셀의 연산이 필요하게 된다.

incoming 값의 재계산? 2-pass?

- **1안:** 메모리를 아끼는 대신 계산을 중복한다. (1-pass)
- **2안:** 계산을 아끼는 대신 작은 중간 버퍼와 barrier 비용을 지불한다. (2-pass)

설계 기본안은 2-Pass + α 저장, 그리고 성능 비교용으로 1-Pass 재계산 버전?

▼ 1안: 1 Pass (폐기)

```
i
├ decay[i]
├ i → 주변 8개 raw flux
├ alpha[i]
├
├ j0 → i 계산
│   └─ alpha[j0] 재계산
│       └─ j0 → 주변 8개 계산
├
├ j1 → i 계산
│   └─ alpha[j1] 재계산
│       └─ j1 → 주변 8개 계산
├
├ ...
├
└─ next[i]
```

- 각 **i** 스레드가 모든 걸 혼자 해결
- 장점은 **중간 버퍼도 없고 barrier도 없다는 것.**
- 대신 **alpha[j]** 하나를 알아내기 위해 **j** 주변의 flux 8개를 매번 다시 계산해야 해서 중복 계산이 상당히 많아진다.

▼ 2안: 2 Pass

먼저 각 texel의 **alpha** 만 한 번 계산한다.

```
Pass 1

i
├ decay
├ raw flux 8개
├ outgoing sum
└─ alpha[i] 저장
```

그다음 barrier.

```
Pass 1
  ↓
alpha 완성
  ↓
Barrier
  ↓
Pass 2
```

Pass 2에서는:

```
j → i
rawFlux(j → i) 재계산
      ×
alpha[j] 읽기
      ↓
실제 flux
```

이제 `alpha[j]` 를 구하려고 `j` 주변 8개를 다시 검사할 필요가 없다.